

Universidad Autónoma de Baja California

Facultad de Ciencias Químicas e Ingeniería



Proyecto de Carrera

3.1 Documentación terminada

Docente: Mauricio Alonso Sanchez Herrera

Alumno:

- Baldenegro De la Vega Jose Angel
- 1288800

Tijuana, Baja California a 29 de Mayo de 2026

Problemática.....	3
Solución propuesta.....	3
Requerimientos del Proyecto.....	4
Requerimientos FURPS.....	4
Requerimientos de Hardware.....	6
Componentes y Herramientas por utilizar.....	8
Componentes de Hardware.....	8
Herramientas de Software.....	9
Funcionamiento del sistema.....	10
Estructura del Proyecto.....	10
Unidad Principal de Control y Comunicación.....	10
Unidad de Captura Visual.....	11
Interconexión entre Módulos.....	12
Prototipos.....	13
Interfaces de Usuario.....	13
Pantalla de Bienvenida (WelcomeScreen).....	13
Pantalla Principal del Mapa de Monitoreo (HomeMapScreen).....	13
Pantalla de Unidades Favoritas (FavoritesScreen).....	15
Modelado 3D del sistema.....	16
Unidad Principal de Control y Comunicación.....	16
Unidad de Captura Visual.....	16
Esquemático.....	17
Módulo Completo.....	17
Resultados.....	18
Evaluación general de los requerimientos.....	18
Integración de hardware y modelado 3D.....	18
Telemetría y sincronización en la nube.....	18
Rendimiento de la aplicación móvil.....	18
Limitaciones técnicas identificadas.....	19
Conclusiones.....	19

Problemática

El transporte público en la ciudad presenta varios problemas relacionados con la falta de control y seguimiento en su operación. Actualmente, no hay un sistema confiable que permita saber si las unidades están cumpliendo con sus rutas, respetando horarios o manteniendo condiciones adecuadas de seguridad.

Para los usuarios, esto se traduce en incertidumbre al momento de trasladarse. No saben con exactitud dónde se encuentra el transporte, cuánto tiempo tardará en llegar o qué tan lleno viene, lo que provoca tiempos de espera largos y una experiencia poco eficiente.

Por otro lado, las autoridades tampoco cuentan con información clara y en tiempo real para supervisar el servicio. Esto dificulta detectar fallas, como desvíos de ruta o irregularidades en la operación, y limita la posibilidad de tomar decisiones basadas en datos.

En conjunto, esta situación afecta la movilidad de la ciudad, la seguridad de los usuarios y la calidad general del servicio de transporte público.

Solución propuesta

Como solución, se propone implementar un sistema embebido que permita monitorear las unidades de transporte en tiempo real y mejorar tanto la experiencia del usuario como la supervisión del servicio.

El sistema integra diferentes tecnologías. Por un lado, utiliza GPS para conocer la ubicación exacta de cada unidad. También incluye un mecanismo para detectar desvíos de ruta, comparando el recorrido real con el establecido. Además, calcula el tiempo estimado de llegada (ETA) para informar a los usuarios de manera más precisa.

Otro componente importante es el conteo de pasajeros mediante visión artificial, procesado directamente en el dispositivo, permitiendo estimar el nivel de ocupación sin perturbar la privacidad.

La información se envía a la nube a través de red celular y se visualiza en una aplicación móvil con mapa interactivo. Desde ahí, los usuarios y autoridades pueden consultar la ubicación en tiempo real, los tiempos de llegada, el nivel de ocupación y recibir alertas ante desvíos o incidencias.

Con este sistema se busca hacer el transporte más eficiente, seguro y transparente, además de brindar herramientas útiles tanto para los usuarios como para las autoridades encargadas de su gestión.

Requerimientos del Proyecto

Requerimientos FURPS

F - Funcionalidad (Functionality)

1. El sistema mostrará en un mapa la ubicación actual de cada unidad de transporte.
2. La posición de cada unidad se actualizará automáticamente cada 30 segundos mientras el vehículo esté en operación.
3. El sistema registrará la velocidad del vehículo en km/h usando datos del módulo GPS.
4. Cada unidad tendrá registrada su ruta habitual como una secuencia de coordenadas geográficas.
5. El sistema comparará la ubicación actual con la ruta registrada para verificar que el vehículo siga su trayecto normal.
6. Se generará una alerta de desvío cuando la unidad se aleje más de 100 metros de su ruta durante al menos 30 segundos.
7. El sistema calculará el tiempo estimado de llegada a un punto seleccionado usando la posición actual, velocidad y ruta registrada.
8. El tiempo estimado se mostrará en minutos dentro de la aplicación móvil.
9. El tiempo estimado de llegada se actualizará automáticamente con cada nueva posición.
10. El sistema estimará la cantidad de pasajeros mediante procesamiento local de imágenes en el módulo de visión.
11. El número de pasajeros se actualizará cada 30 segundos.
12. Solo se transmitirá el número de pasajeros, sin guardar imágenes ni datos personales.
13. Los datos de posición, velocidad, ocupación y estado del sistema se enviarán a la nube mediante red celular.
14. Si no hay conexión, los datos se guardarán temporalmente en el dispositivo y se enviarán cuando se recupere la señal.
15. La información en la nube se sincronizará casi en tiempo real para que los usuarios vean datos actualizados.

U - Usabilidad (Usability)

1. La aplicación móvil mostrará la información en un mapa interactivo con marcadores diferenciados por tipo de unidad.
2. La interfaz funcionará correctamente tanto en computadora como en dispositivos móviles desde el navegador.
3. Cada unidad mostrará un panel de información con:
 - a. ubicación
 - b. ocupación estimada
 - c. tiempo estimado de llegada

4. Los colores, íconos y alertas permitirán identificar rápidamente el estado de cada unidad.
5. El uso de la aplicación se limitará a acciones simples como seleccionar o filtrar unidades, sin requerir conocimientos técnicos.

R - Confiabilidad (Reliability)

1. El sistema seguirá operando aunque existan fallas temporales de señal celular, almacenando datos localmente.
2. El módulo de comunicación intentará reconectarse automáticamente cada 15 segundos.
3. Los datos generados durante fallas de conexión no se perderán.
4. El sistema podrá operar durante largos periodos sin reinicios manuales.
5. El hardware funcionará de forma estable dentro del vehículo pese a vibraciones, variaciones de voltaje y cambios de temperatura.

P - Rendimiento (Performance)

1. Cada paquete de datos se generará y enviará en un máximo de 5 segundos desde su obtención.
2. El tiempo estimado de llegada se calculará antes de la siguiente actualización de posición.
3. El conteo de pasajeros se realizará localmente sin afectar la obtención de ubicación ni el envío de datos.
4. El consumo energético del sistema se mantendrá por debajo de 5 W en operación normal.
5. La aplicación móvil mostrará la información con un retraso máximo de 3 segundos desde que los datos llegan a la nube.

S - Soportabilidad (Supportability)

1. El sistema permitirá agregar nuevas unidades mediante configuración sin modificar el hardware existente.
2. Los módulos de posicionamiento, comunicación y visión podrán actualizarse o reemplazarse de forma independiente.
3. El software del microcontrolador podrá actualizarse sin desmontar el equipo del vehículo.
4. Se podrán realizar pruebas o diagnósticos por unidad sin afectar el funcionamiento de las demás.
5. La base de datos estará organizada para facilitar mantenimiento, consultas y crecimiento del sistema.

Requerimientos de Hardware

1. Unidad Central de Control

Microcontrolador principal:

- Debe ser un dispositivo embebido con capacidad de conectividad, procesamiento y bajo consumo energético.
- Debe incluir soporte para interfaces de comunicación serie que permitan conectar módulos externos.

2. Sensores y Módulos de Captura de Datos

Módulo de posicionamiento satelital:

- Requerido para obtener la latitud, longitud y velocidad del vehículo.
- Debe tener compatibilidad con señales GPS y ofrecer precisión adecuada para uso vehicular.
- Debe funcionar con un rango de voltaje estándar para sistemas embebidos.

Módulo de conectividad celular:

- Esencial para transmitir datos desde la unidad de transporte a la red, incluso sin puntos de acceso Wi-Fi.
- Debe soportar transmisión de datos mediante redes móviles y ser compatible con interfaces de texto simple para control y envío de información.

Módulo de visión artificial para conteo de pasajeros:

- Necesario para capturar imágenes y procesarlas localmente con el fin de estimar la cantidad de personas dentro del transporte.
- Debe integrar una cámara y capacidad de procesamiento suficiente para análisis de imágenes sin sobrecargar el sistema principal.
- Este módulo se comunica con la unidad central para enviar únicamente los resultados de conteo.

3. Fuente de Alimentación

El hardware debe incluir un sistema de alimentación estable y adecuado para uso automotriz:

- Una fuente primaria puede obtener energía del sistema eléctrico del vehículo.
- Debe utilizar reguladores de voltaje que conviertan la energía del vehículo a los niveles requeridos por cada componente del sistema.
- Es importante que la alimentación sea robusta frente a variaciones de voltaje propias de vehículos en movimiento.

4. Comunicación entre Componentes

- Los módulos de localización, visión y conectividad deben comunicarse eficientemente con la unidad de control.
- Se debe contar con comunicación serial u otra interfaz estándar entre los módulos para garantizar la transferencia de datos sin pérdida.

5. Interconexión Física

- Todo el conjunto de hardware debe montarse de manera segura dentro del vehículo, con conexiones eléctricas protegidas contra vibraciones y movimientos.
- Los componentes deben colocarse de forma que no interfieran con el funcionamiento normal del transporte.

6. Antenas

- El sistema puede incluir antenas apropiadas para:
 - Recepción de señal satelital (GPS).
 - Conectividad celular estable.
- La colocación de las antenas debe permitir buena recepción sin obstrucciones.

Componentes y Herramientas por utilizar

Componentes de Hardware

Componente	Función Principal	Razón de uso	Voltaje de Operación	Precio EST. (USD)
ESP32	Coordinar el sistema, decodificar la información del GPS, administrar el almacenamiento local de respaldo y transmitir los datos a la nube.	Ofrece conectividad Wi-Fi integrada, múltiples puertos seriales (UART) de hardware para comunicar los módulos y soporte para el sistema de archivos LittleFS a bajo costo.	3.3 V	\$4.50 - \$6.00
ESP32-CAM	Capturar imágenes del interior del transporte y procesar localmente el algoritmo de conteo de personas.	Integra un sensor de cámara (OV2640), cuenta con la memoria RAM suficiente para ejecutar redes neuronales ligeras en el borde (Edge AI) y admite el modo de bajo consumo Deep Sleep.	5 V	\$7.00 - \$9.00
NEO-6M	Recibir señales satelitales para calcular la latitud, longitud y velocidad en tiempo real de la unidad.	Proporciona datos de localización con precisión vehicular, transmite información mediante protocolo serial estándar (NMEA) y cuenta con una antena pasiva de fácil colocación.	2.7 V a 3.6 V	\$7.00 - \$9.00

Herramientas de Software

Herramienta / Librería	Tipo de Software	Función Principal
Flutter	Framework de Desarrollo	Permite compilar la aplicación móvil multiplataforma para el usuario final utilizando un esquema visual modular.
Firebase Realtime Database	Base de Datos NoSQL en la Nube	Almacena y distribuye centralizadamente la telemetría de los camiones, notificando al instante los cambios a la aplicación móvil.
TinyGPSPlus	Librería de Firmware (C++)	Decodifica en segundo plano las sentencias NMEA nativas enviadas por el módulo NEO-6M para extraer coordenadas y velocidad limpia.
LittleFS	Sistema de Archivos Embebido	Administra la memoria flash interna del ESP32 para crear, leer y añadir registros en el archivo local durante fallas de red.
Red Neuronal MTMN	Framework de Visión Artificial	Ejecuta de forma local en la ESP32-CAM la detección de estructuras faciales para el conteo de pasajeros, cuidando la privacidad del usuario.
google_maps_flutter	Complemento/Plugin de Flutter	Integra la cartografía interactiva de Google Maps dentro de la aplicación móvil y renderiza los marcadores dinámicos de los camiones.
Geolocator	Complemento/Plugin de Flutter	Obtiene la ubicación GPS del teléfono del usuario y calcula la distancia lineal entre este y la unidad para la posterior estimación del ETA.
SharedPreferences	Sistema de Almacenamiento Local	Guarda de forma permanente en la memoria del teléfono los IDs de las unidades que el usuario ha marcado como favoritas.

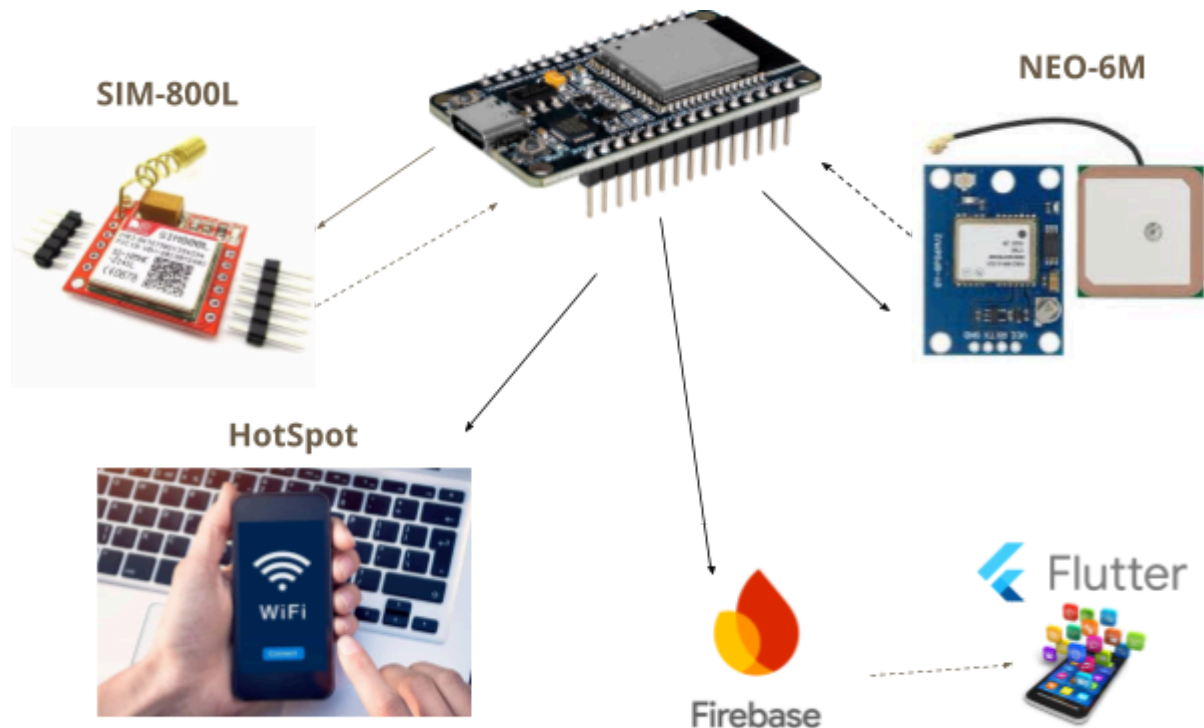
Funcionamiento del sistema

Estructura del Proyecto

El sistema está diseñado bajo un modelo de arquitectura distribuida que separa las tareas de telemetría y comunicación de las tareas de visión artificial pesadas. El ecosistema se compone de dos nodos embebidos independientes que cooperan de forma asíncrona:

- **Nodo Central (ESP32):** Actúa como el maestro del sistema. Gestiona la adquisición de datos satelitales, el almacenamiento local de contingencia, la reconexión de red y la sincronización en tiempo real con la base de datos en la nube (Firebase Realtime Database).
- **Co-procesador de Visión (ESP32-CAM):** Dedicado exclusivamente a la captura y conteo de pasajeros en el borde (Edge AI), comunicándose de forma unidireccional con el nodo central.

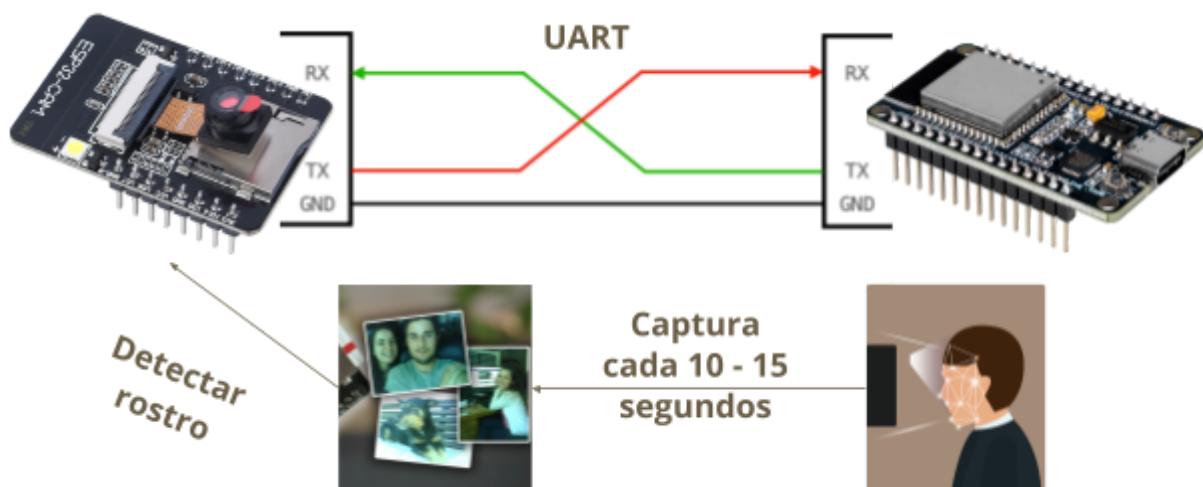
Unidad Principal de Control y Comunicación



La unidad principal ejecuta un ciclo de control basado en hilos virtuales simulados por temporizadores no bloqueantes (`millis()`), lo que garantiza que el procesamiento de datos de un módulo no atranque las funciones de los demás.

- **Lectura de Posicionamiento:** A través del puerto UART2, el microcontrolador recibe constantemente las sentencias NMEA nativas del módulo GPS NEO-6M a 9600 bps. La librería TinyGPSPlus decodifica estas sentencias en segundo plano para mantener actualizadas las variables de latitud, longitud y velocidad (en km/h).
- **Transmisión Periódica y Formato de Datos:** Cada 30 segundos exactos, el sistema compila un objeto JSON estructurado con la identificación de la unidad ("012"), la ruta ("Universitaria"), el color del vehículo, las coordenadas espaciales, la velocidad y el último aforo registrado. Este paquete se envía mediante una petición directa a la ruta jerárquica en Firebase.
- **Mecanismo de Resguardo Local (LittleFS):** En caso de detectar una desconexión de la red Wi-Fi o una falla de respuesta de Firebase, el sistema activa su protocolo de resiliencia: los datos actuales no se descartan, sino que se formatean en una línea de texto plano y se añaden (FILE_APPEND) al archivo /offline.csv almacenado en la memoria flash interna mediante el sistema de archivos LittleFS.
- **Vaciado de Caché (Sincronización):** Cuando la conectividad se restablece con éxito, antes de enviar el dato actual, el ESP32 abre el archivo offline, lee secuencialmente los paquetes atrasados y los impacta en la nube bajo la etiqueta "tipo_envio": "recuperado_cache". Una vez asegurado el envío de cada línea, limpia la memoria local para optimizar el almacenamiento del chip.

Unidad de Captura Visual



Para mitigar problemas de consumo de energía (menor a los 5 W requeridos) y sobrecalentamiento, la ESP32-CAM opera bajo un esquema cíclico gobernado por hardware mediante Deep Sleep:

- **Rutina de Encendido Único:** El módulo no ejecuta código en el bucle infinito (loop). Al arrancar, inicializa el sensor de la cámara con una resolución optimizada de QQVGA (160x120 píxeles) en formato RGB565. Esta configuración reduce drásticamente el tamaño del mapa de bits en memoria RAM, acelerando el procesamiento matemático.
- **Inferencia en el Extremo (Edge AI):** La imagen se transforma a formato RGB888 y es procesada inmediatamente por la red neuronal multitarea MTMN configurada localmente. El algoritmo busca estructuras faciales basándose en un umbral de puntuación estricto para evitar falsos positivos provocados por el movimiento del transporte.
- **Salida de Datos y Apagado Térmico:** Tras realizar la detección, el co-procesador imprime en su puerto serial la cadena formateada `FACES:[número]`, libera por completo la memoria dinámica utilizada por la matriz de la imagen y entra en un estado de sueño profundo durante 30 segundos. Durante este tiempo, el consumo eléctrico del módulo cae a microamperios, protegiendo los componentes automotrices.

Interconexión entre Módulos

La transferencia interna de datos se consolida a nivel de hardware utilizando enlaces punto a punto mediante buses seriales (UART) independientes, aislándolos de colisiones de datos:

- **Enlace GPS-ESP32:** Conectado a los pines GPIO 16 (RX) y 17 (TX) a través de `HardwareSerial(2)`.
- **Enlace ESP32_CAM-ESP32:** Configurado a través de `HardwareSerial(1)` utilizando el pin GPIO 4 (RX) del ESP32 principal. Este enlace es estrictamente unidireccional (recepción en el maestro), lo que optimiza el uso de pines en la placa central.
- **Resguardo de Conectividad:** Aunque el diseño de hardware contempla la distribución de pines para un módem celular SIM800L (pines 26 y 27), la capa de transporte actual se consolidó mediante el módulo Wi-Fi nativo del ESP32, utilizando un punto de acceso móvil inalámbrico para la salida hacia el internet comercial.

Prototipos

Interfaces de Usuario

La aplicación móvil, desarrollada bajo el framework Flutter, actúa como la capa de presentación y visualización del sistema para los usuarios finales. Está construida bajo un enfoque modular mediante componentes reutilizables y pantallas reactivas que consumen la información en tiempo real directamente desde la base de datos distribuida.

Pantalla de Bienvenida (WelcomeScreen)

Es la pantalla de inicio de la aplicación. Presenta una interfaz limpia y minimalista con la identidad visual del proyecto ("Smart Transit"). Su única función es dar acceso al usuario y redirigirlo hacia la pantalla principal del mapa.

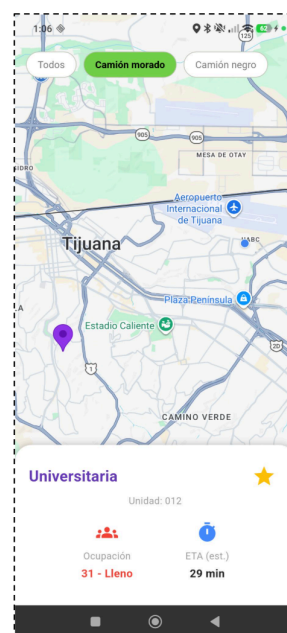
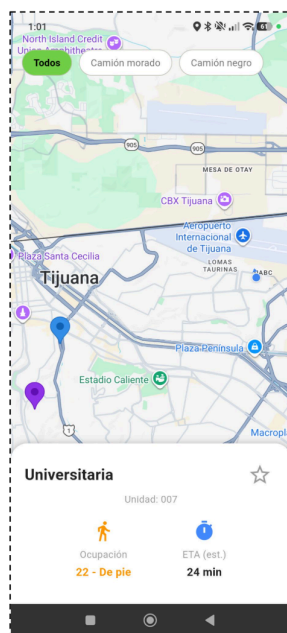
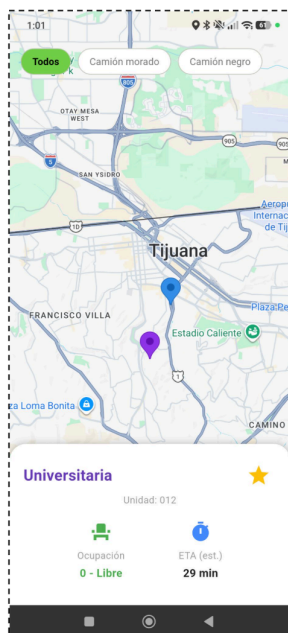


Pantalla Principal del Mapa de Monitoreo (HomeMapScreen)

Esta sección centraliza la lógica interactiva y el procesamiento de la telemetría que se recibe desde la base de datos. Su funcionamiento se basa en las siguientes características:

- **Suscripción de Datos en Tiempo Real:** En lugar de realizar consultas periódicas que saturan la red, la aplicación mantiene una conexión continua mediante un objeto `StreamSubscription` apuntado al nodo de `Firebase Realtime Database`. Cada vez que un camión actualiza su ubicación (cada 30 segundos), `Firebase` notifica el cambio y la aplicación refresca la interfaz automáticamente (`setState`).

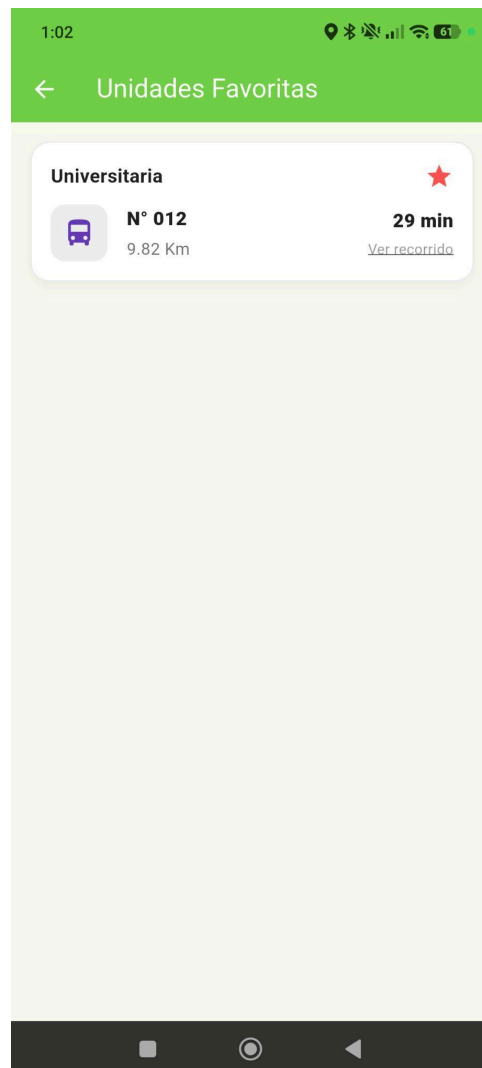
- **Renderizado del Mapa Interactiva:** Utiliza la API de Maps_flutter para mostrar la ubicación de los camiones sobre el mapa urbano. Los marcadores cambian de color de forma dinámica según el tipo de unidad seleccionada ("Camión morado" o "Camión negro").
- **Cálculo Local de Distancia y ETA:** Como el servidor no cuenta con un trazado de rutas geográficas exactas, la aplicación calcula el tiempo estimado de llegada de forma local en el teléfono utilizando la librería Geolocator:
 - Primero, obtiene la distancia en línea recta entre la ubicación GPS del usuario y las coordenadas actuales del camión.
 - **Factor de Ajuste Urbano:** Para aproximar este cálculo a la distancia real de las calles, la distancia en línea recta se multiplica por un factor de corrección de 1.4.
 - **Cálculo de Tiempo:** Asumiendo una velocidad promedio de 20 km/h (común para el transporte público en zonas urbanas), la aplicación traduce la distancia estimada a minutos y actualiza el indicador en pantalla.
- **Panel de Información Desplegable:** Al tocar un marcador en el mapa, se despliega un panel inferior con la información detallada de la unidad a través de tarjetas modulares (InfoCard). Este panel utiliza un código de colores muy claro para indicar el nivel de ocupación detectado previamente por la ESP32-CAM:
 - **Menos de 15 pasajeros:** Estado Libre (Verde / Ícono de asiento).
 - **Entre 15 y 30 pasajeros:** Estado De pie (Naranja / Ícono de peatón).
 - **Más de 30 pasajeros:** Estado Lleno (Rojo / Ícono de grupo).



Pantalla de Unidades Favoritas (FavoritesScreen)

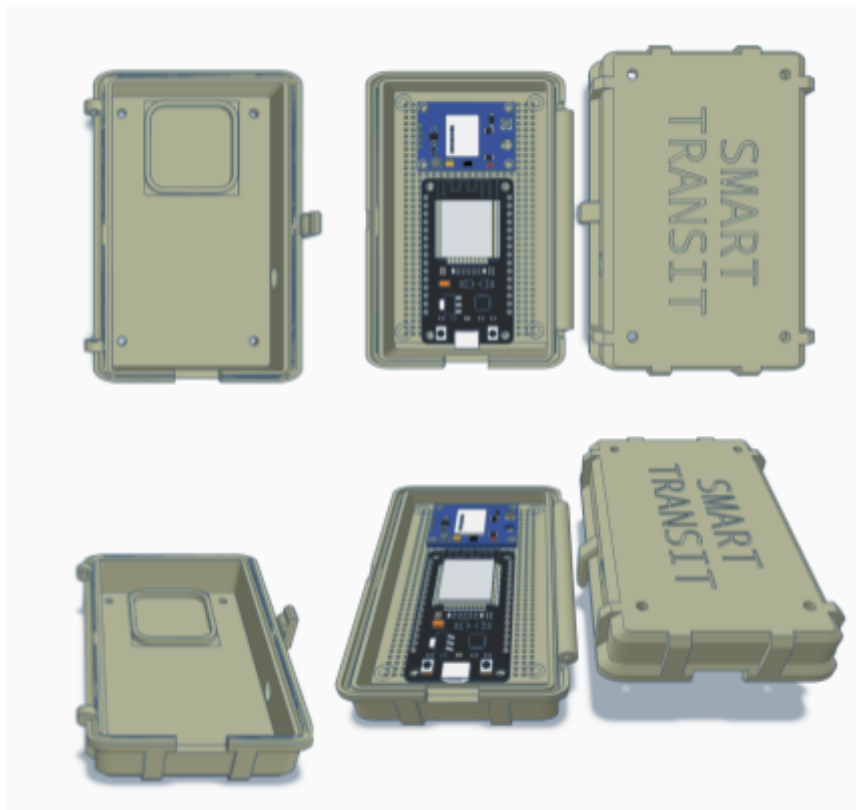
Para que los usuarios puedan consultar rápidamente sus camiones más frecuentes, se implementó un sistema de almacenamiento local utilizando **SharedPreferences**.

- **Persistencia por ID:** Cuando el usuario presiona el botón de estrella en el panel principal, el ID de la unidad se guarda de forma permanente en la memoria del teléfono.
- **Filtrado Activo y Diagnóstico:** Al entrar a la sección de favoritos, la aplicación contrasta los IDs guardados con los camiones activos en Firebase. Si una unidad favorita no está transmitiendo datos, la interfaz lo detecta y muestra una tarjeta gris con la alerta "Fuera de servicio o sin señal". Si la unidad está activa, la pantalla calcula en masa la distancia y el ETA de cada camión guardado, mostrándolos en una lista ordenada.

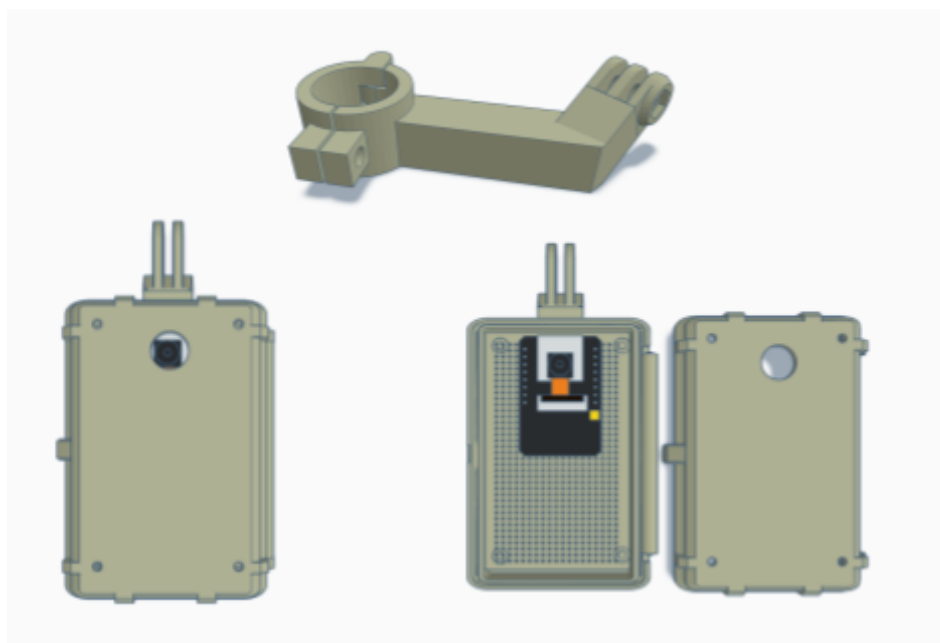


Modelado 3D del sistema

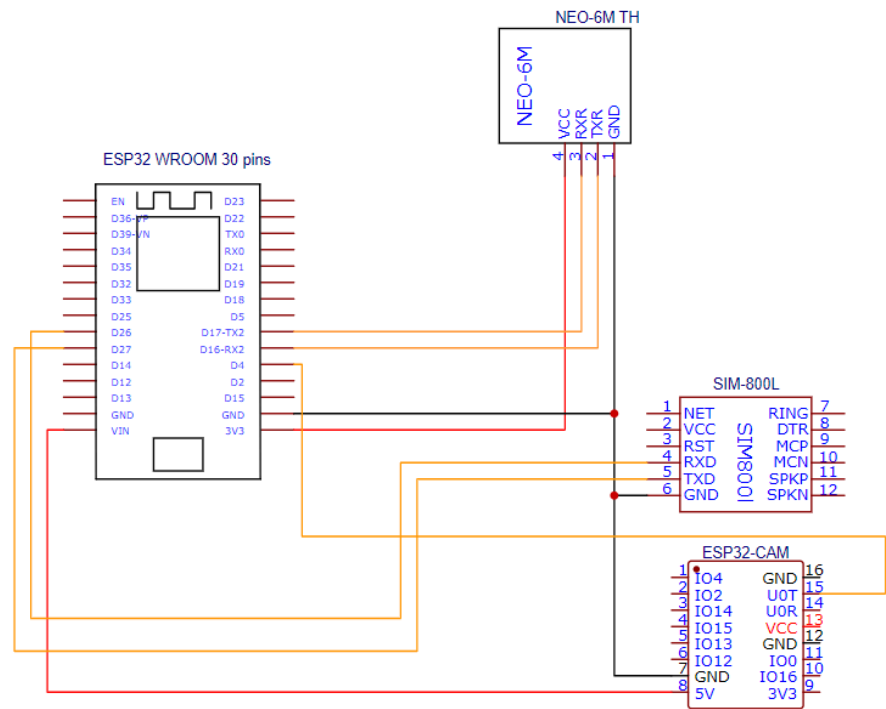
Unidad Principal de Control y Comunicación



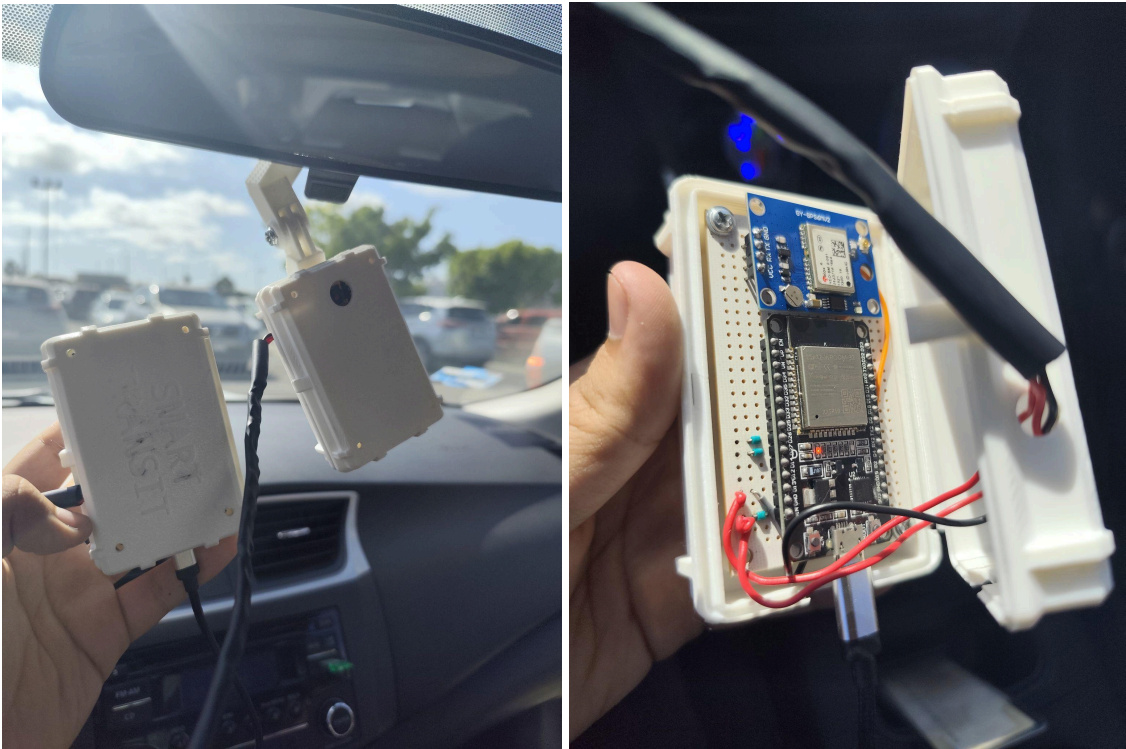
Unidad de Captura Visual



Esquemático



Módulo Completo



Resultados

Evaluación general de los requerimientos

Una vez concluidas las fases de ensamblaje, programación y pruebas del sistema, se evaluó el cumplimiento de los requerimientos planteados al inicio del proyecto. El prototipo final demostró un excelente desempeño en sus funciones principales de telemetría, integración de hardware y visualización en tiempo real. Asimismo, las pruebas de campo permitieron identificar con claridad las limitaciones técnicas de los módulos de geolocalización avanzada y visión artificial.

Integración de hardware y modelado 3D

El diseño y la construcción del prototipo físico cumplieron con éxito los estándares establecidos:

- **Montaje y soldadura:** Todos los componentes electrónicos se soldaron correctamente en placas de desarrollo. Esto garantizó una conexión eléctrica estable y resistente a las vibraciones normales dentro de un vehículo.
- **Gabinete físico:** El modelo 3D diseñado para proteger los circuitos funcionó perfectamente. Las dimensiones de la pieza impresa coincidieron con la distribución real de los componentes, lo que permitió un ensamble limpio y una colocación segura.

Telemetría y sincronización en la nube

- **Lectura GPS:** El módulo NEO-6M, en conjunto con la librería TinyGPSPlus, obtuvo con precisión las coordenadas geográficas y calculó la velocidad del vehículo en tiempo real.
- **Enlace con Firebase:** La comunicación con Firebase Realtime Database operó sin interrupciones, logrando actualizar y sincronizar la ubicación en la nube puntualmente cada 30 segundos.

Rendimiento de la aplicación móvil

- **Despliegue de datos:** La interfaz mostró la telemetría directamente desde la base de datos sin presentar retrasos ni caídas en el servicio.

- **Interactividad:** El mapa interactivo proyectó correctamente la ubicación de las unidades, y los paneles desplegables organizaron la información de forma clara y accesible para el usuario final.

Limitaciones técnicas identificadas

Durante las pruebas de campo, se detectaron dos problemas críticos que impidieron cumplir con la totalidad de los requerimientos funcionales:

- **Verificación de ruta y alertas de desvío:** Debido a las limitaciones de memoria interna en el ESP32 para almacenar listas extensas de coordenadas geográficas, no fue posible guardar el trayecto completo de la ruta de forma local. Por esta razón, quedaron pendientes de implementar la comparación de la ruta en tiempo real y la activación de alertas automáticas si el vehículo se alejaba más de 100 metros.
- **Eficiencia en el conteo de pasajeros:** La red neuronal MTMN ejecutada en la ESP32-CAM tuvo un rendimiento bajo en el entorno real, logrando detectar rostros únicamente en el 5% de las pruebas. Este bajo porcentaje se debió a factores del entorno difíciles de controlar, como los cambios bruscos de luz dentro del vehículo, el movimiento constante de la cámara por las vibraciones y la baja resolución de imagen que se tuvo que usar para no saturar la memoria RAM del chip.

Conclusiones

El desarrollo de este proyecto demostró que es totalmente viable diseñar y construir un sistema de telemetría de bajo costo para el transporte público utilizando tecnologías de Internet de las Cosas (IoT) y aplicaciones multiplataforma. La combinación de hardware embebido con bases de datos en tiempo real demostró ser una solución muy eficiente para reducir la incertidumbre de los usuarios, permitiéndoles conocer la ubicación y velocidad exacta de los vehículos con gran estabilidad.

Aunque las limitaciones de procesamiento en los circuitos del vehículo impidieron que el conteo de pasajeros y la detección de desvíos funcionaran de forma óptima, la estructura principal del sistema operó sin fallas. El proyecto deja una base técnica sólida y demuestra que la arquitectura propuesta es escalable y lo suficientemente resistente para soportar las condiciones de uso automotriz.